

סיכום שלבי העבודה, ההתפתחות וההיגיון המדעי בפרויקט

פרויקט: זיהוי בוטים בתנועות עכבר באמצעות גאומטריה יישומית וקינמטיקה (Kinematic Bot Detection)
קורס אקדמי: גאומטריה יישומית (Applied Geometry)
מטרה: תיעוד רציף של שבעת שלבי הפיתוח, האתגרים שעלו בכל שלב, והרציונל המדעי שהוביל לשלב הבא.

[שלב 3: מנוע גאומטריה ועקמומיות] → [שלב 2: בוטים נאיביים] → [שלב 1: איסוף ראשוני]
|
[שלב 5: ביומכניקה ובוט מתחזה] ← [שלב 6: שבירת ה-100%] ← [PDF שלב 7: דמו חי ודוח]

שלב 1 נקודת ההתחלה – איסוף תנועות אנושיות בסיסיות

מה עשינו בפועל: בנינו ממשק איסוף גרפי מבוסס Tkinter (mouse_tracker.py). על משטח הציור בגודל 900×600 פיקסלים, המערכת מגרילה בכל ניסיון מעגל התחלה ירוק ומעגל יעד אדום. המשתמש לוחץ על הירוק, מזיז את העכבר ליעד, והמערכת דוגמת קואורדינטות וזמנים $\mathcal{T} = \{(x_i, y_i, t_i)\}_{i=1}^N$ ונשמרת כקובץ CSV ב- data/session_01/.

ההיגיון והמניע המדעי: הצעד הראשון בכל מחקר ביומטרי מבוסס למידת מכונה הוא יצירת נתוני אמת של משתמשים אנושיים (Ground Truth Data). היה צורך להקליט תנועות יד טבעיות מגוונות (בכיוונים, מרחקים וזוויות שונות) כדי להבין את מאפייני התנועה האנושית הבסיסית.

הבעיה והמגבלה שעלתה מיד: במאגר הנתונים היו אך ורק מסלולים של בני אדם. לא ניתן לבנות או לבחון מסווג בינארי ללא דוגמאות של הצד השני (בוטים). אם אין בוטים סינתטיים להשוואה, אין שום דרך מדעית לבדוק האם גדלים גאומטריים מסוגלים להבדיל בין אדם למחשב.

שלב 2 כניסת מחוללי הבוטים הנאיביים (linear, curved, noisy)

מה עשינו בפועל: הוספנו ישירות ל-GUI כפתורים המאפשרים לבצע תנועה אוטומטית בין המעגלים:

- bot_linear: תנועה בקו ישר בדיוק בין המרכזים במהירות קבועה לחלוטין ($u = \tau$).
- bot_curved: תנועה לאורך קשת בזייה ריבועית (Quadratic Bézier) עם נקודת בקרה בניצב למיתר במהירות אחידה.
- bot_noisy: תנועה בקו ישר עם תוספת רעש צידי רציף (אינטרפולציית הרמיט מסוג Smoothstep: $3\tau^2 - 2\tau^3$) ומעטפת דעיכה בנקודות הקצה.

ההיגיון והמניע המדעי: יצירת קבוצת ביקורת בסיסית (Baseline) שתאפשר לבחון מדדים גאומטריים פשוטים: יחס אורך המסלול למרחק הישיר ($\text{Path Ratio} = L/\text{direct}$) וסטיית מיתר מקסימלית ($\max_chord_dev_px$).

הממצא והמגבלה שחשפנו: הבוטים הנאיביים נעים כולם במהירות קבועה לחלוטין (יחס שיא המהירות לממוצע ≈ 1.0). אמנם קל מאוד לזהות אותם, אך בעולם האמיתי תוקף מתוחכם לא ישתמש בבוט שנע במהירות קבועה ובמסלול פשוט. היה ברור שעלינו לבנות מנוע ניתוח גאומטרי מעמיק הנשען על גאומטריה דיפרנציאלית וקינמטיקה אמיתית.

מה עשינו בפועל: בנינו מנוע אנליטי שמחלץ את הצורה הגאומטרית הטהורה של המסלול במנותק מהזמן:

1. פרמטריזציה לפי אורך קשת (Arc-Length Resampling): חישוב אורך הקשת המצטבר s_k ודגימה מחדש ל-100 נקודות במרווח מרחבי אחיד $ds = L/99$ כדי שוקטור המשיק יקיים $\|\gamma'(s)\| = 1$.

2. פתרון לבעיית הרעש בגזירה בדידה (Cubic Savitzky-Golay): גזירה בדידה של הפרשים סופיים על רשת פיקסלים שלמים ($\mathbb{Z}^2$) הגבירה את רעש הדיגיטציה ביחס הפוך ל- ds^2 . הפתרון: התאמת פולינום מעלה 3 בחלון נע של 7 נקודות באמצעות ריבועים פחותים, וגזירה אנליטית במרכז החלון לקבלת עקמומיות חלקה מדרגה C^2 .

$$\kappa = \frac{2|a_1b_2 - b_1a_2|}{(a_1^2 + b_1^2)^{3/2}}$$

3. מנגנון מסכות מקומיות (Local Exclusion Masks): החרגת שולי קצוות ($span/2$), החרגת מרווחים דלילים הנובעים מדילוגי חומרה ($segment_lengths > span$), והחרגת היפוכי כיוון חדים (180°) למניעת עקמומיות אינסופיות בנקודות חורפה (Cusps).

ההיגיון המדעי: יישום ישיר ומדויק של החומר הנלמד בקורס: מסגרת פרנה-סרה, אינווריאנטיות לטרנספורמציות קשיחות (סיבוב והזזה), ואפיון עקומות מישוריות בדידות.

מה עשינו בפועל:

• בוט Min-Jerk (`bot_smart_jerk`): יישום פתרון משוואת אוילר-לגראנז' של מודל Minimum Jerk (Flash & Hogan, 1985 – פולינום מעלה 5:

$$u(\tau) = 10\tau^3 - 15\tau^4 + 6\tau^5, \quad \tau = \frac{t}{T}$$

הפולינום יצר פרופיל מהירות פעמוני חלק ומדויק (האצה ובלימה הדרגתית).

• בוט ביומכני (`bot_smart_full`): שילוב של פרופיל א-סימטרי המגיע לשיא מוקדם באמצעות חזקה $\tau^{0.8}$, הארכת שלב הבלימה האופיינית ליד אנושית, ורעד נורו-מוטורי פיזיולוגי בשני תדרים אמיתיים (10Hz ו-18Hz).

• אמידת חוק שני-השלישים (Two-Thirds Power Law): חילוץ מעריך החוק β במשוואת הרגרסיה $\ln v = C - \beta \ln \kappa$, וחישוב מקדם ההשתנות של המהירות האפינית: $v_{aff} = \kappa^{1/3}$.

ההיגיון והממצא המדעי: הוכחנו אמפירית כי אצל בני אדם קיים צימוד הדוק בין עקמומיות למהירות ($\beta \approx 0.39$, קרוב מאוד לערך התיאורטי $1/3$), בעוד אצל כל הבוטים (כולל בוט ה-Min Jerk) המעריך היה קרוב לאפס ($\beta \approx 0.0$). המודל למד להפריד בין הבוטים החכמים לבני אדם בזכות העובדה שהבוטים לא צימדו בין העקמומיות למהירות.

התפנית – "הבוט לא מספיק חזק, אני עדיין מצליחה לרמות את המודל!"

האתגר שעלה מהשטח: במהלך ניסויים חיים שביצעה המשתמשת עם הממשק, עלתה הבחנה קריטית: תנועות אנושיות מסוימות – בפרט תנועות "הצלפה" מהירות ועקומות (Flicks) – הצליחו לבלבל את המודל או להיות מסוגלות בטעות כבוטים. בנוסף, הבנו שבוט מתקדם שיידע לחקות גם את חוק ה-2/3 עלול להערים על המערכת.

ההיגיון והפתרונות ההנדסיים שפיתחנו:

1. פיתוח בוט מתחזה אדברסריאלי (bot_adversarial): בוט יריב מתוחכם הדוגם עקומת בזייה ומצמד באופן מפורש בין העקמומיות המקומית למהירות המשיקית לפי חוק ה-2/3 $\kappa(s) \propto v(s)^{-1/3}$, תוך שמירה על מעטפת פעמון חלקה.
2. חילוף מדדי גאומטריה מרחבית טהורה:
 - o bezier_residual_px (שארית עקומת בזייה ריבועית): פיתחנו נוסחה אנליטית בריבועים פחותים למציאת נקודת הבקרה הניצבת האופטימלית P_1^* . השארית האוקלידית הממוצעת מעקומת הבזייה היא מזערית בבוטים סינטיים הנעים על נוסחה $(0 - 3px)$, וגבוהה משמעותית באדם $(8 - 25px)$ (עקב תנועת מפרקי היד (כתף, מרפק, שורש כף היד)).
 - o time_to_peak_ratio (תזמון שיא המהירות): אדם מגיע לשיא המהירות מוקדם (ב-25% עד 45% ממשך התנועה), בעוד בוטים סימטריים מגיעים בדיוק ב-50%.
3. שדרוג חישוב ה-Jerk ל-CubicSpline רציף: גזירה בדידה בתנועות אנושיות מהירות (0.2s) גרמה להתאפסות מלאכותית של הנגזרת השלישית. המעבר לספליין קובי פתר את הבעיה ושמר על ג'רק פיזיקלי רציף.

התובנה המדעית הביקורתית – "הדיוקים שלך יותר מדי אידאליים (100%)"

הביקורת המדעית: המשתמשת הבחינה בצדק שדיוק של 100% בסיווג בינארי ו-99.2% בסיווג רב-מחלקתי הוא "אידאלי מדי" ומעורר חשד לזליגת מידע או לפיצ'ר שמבצע הפרדה מלאכותית.

החקירה הסטטיסטית שערכנו: ניתחנו את התפלגויות כל 11 התכונות ומצאנו את הגורם: log_dimensionless_jerk יצר פער עצום ומלאכותי (בוטים: ≤ 10.08 , בני אדם: ≥ 11.65). הסיבה: ביד אנושית קיים רעד שרירי טבעי (Physiological Tremor ב-8-12Hz) ואי-סדירות דגימת USB במערכת ההפעלה. כשגוזרים בדידה 3 פעמים ומחלקים ב- $(\Delta t)^3$, הרעד הזה מקפיץ את אינטגרל הג'רק לסדרי גודל עצומים ($10^{16} - 10^{12}$). לעומת זאת, בוטים מיוצרים מנוסחאות מתמטיות חלקות. עץ ההחלטה פשוט למד את החוק הקל הזה.

הפתרון המדעי – חלוקה ל-3 בנצ'מרקים מחמירים ב-classify_trajectories.py:

- **Benchmark A (מודל מלא עם Jerk):** דיוק 100% – מדגים את כוחו של הרעד הפיזיולוגי.
- **Benchmark B (מודל גיאומטרי טהור ללא Jerk):** דיוק ריאליסטי של 94.7% בעץ החלטה ו-98.5% ביער אקראי – נשען על שארית בזייה, יחס שיא לממוצע ומהירות אפנית.
- **Benchmark C (הכללה בין שניים – Leave-One-Session-Out):** בדיקה על סשנים חדשים לחלוטין (94.0% ממוצע). בסשן 12 הדיוק צנח ל-50.0%. הניתוח האיכותני חשף: תנועות אנושיות מהירות וישרות (Flicks) מתנהגות גיאומטרית ממש כמו בוט מעוקל סינטי.

מה עשינו בפועל:

- מערכת הדגמה אינטראקטיבית בזמן אמת (`Live_demo.py`): ארכיטקטורת In-Memory הפועלת ללא כתיבה לקבצים בדיסק. חישוב כל 11 התכונות בתוך פחות מ-2 מילישניות וסיווג מידי עם כרטיס החלטה והסתברויות מכוילות.
- חבילת בדיקות יחידה מקיפה: 27 בדיקות `unittest` המאמתות גיאומטריה, תקינות בוטים וסיווג ב-100% הצלחה.
- דוח מחקר מלא כקובץ PDF: קימפול מסמך אקדמי בן 13 עמודים (`PROJECT_COMPLETE_REPORT_HE.pdf`) הכולל את כל ההוכחות המתמטיות, הנוסחאות, הטבלאות והגרפים.

🎯 תמצית המסר להגנה האקדמית על הפרויקט (Defense Pitch)

"הפרויקט שלנו לא קפץ לפתרון קסם שחור. התחלנו ממעקב פשוט ובוטים נאיביים. כשהבנו שגזירה בדידה נכשלת עקב קוונטיזציית פיקסלים, פיתחנו התאמת פולינום מעלה 3 בחלון נע מתוך הגאומטריה הדיפרנציאלית. כשראינו שהבוטים נאיביים מדי, יישמנו מודל Minimum Jerk ואת חוק שני-השלישים האפיני. כשהצלחנו לרמות את המודל בתנועות מהירות, בנינו בוט אדברסריאלי והוספנו שארית עקומת בזייה. והכי חשוב – כשקיבלנו 100% דיוק, לא הסתנוורנו: פירקנו את הנתונים, גילינו שרעד החומרה והג'רק יוצרים הפרדה מלאכותית, והוכחנו שהמודל עובד בצורה חזקה ומרשימה (מעל 94%) גם על גאומטריה מרחבית טהורה ועל משתמשים חדשים לחלוטין!"